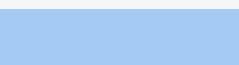
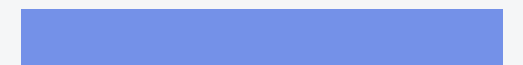
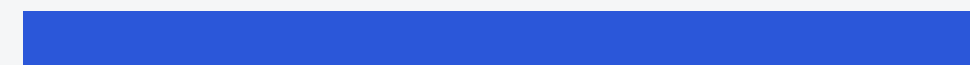
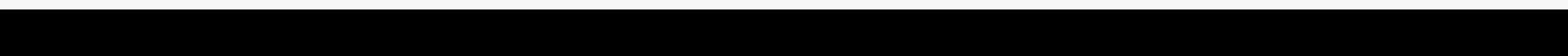


Lecture-5

Functions

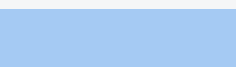
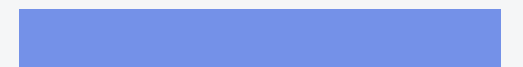
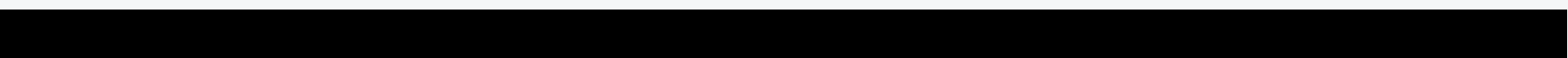


Agenda

01. Declaration, Definition and Call
02. Passing Arguments to Functions
03. Scope and Storage Classes
04. Return statement
05. Recursion
06. Inline Functions
07. Function Overloading

01.

Declaration, Definition and Call



Function

- A function groups program statements into a unit and gives it a name.
- This unit can then be invoked from other parts of the program.

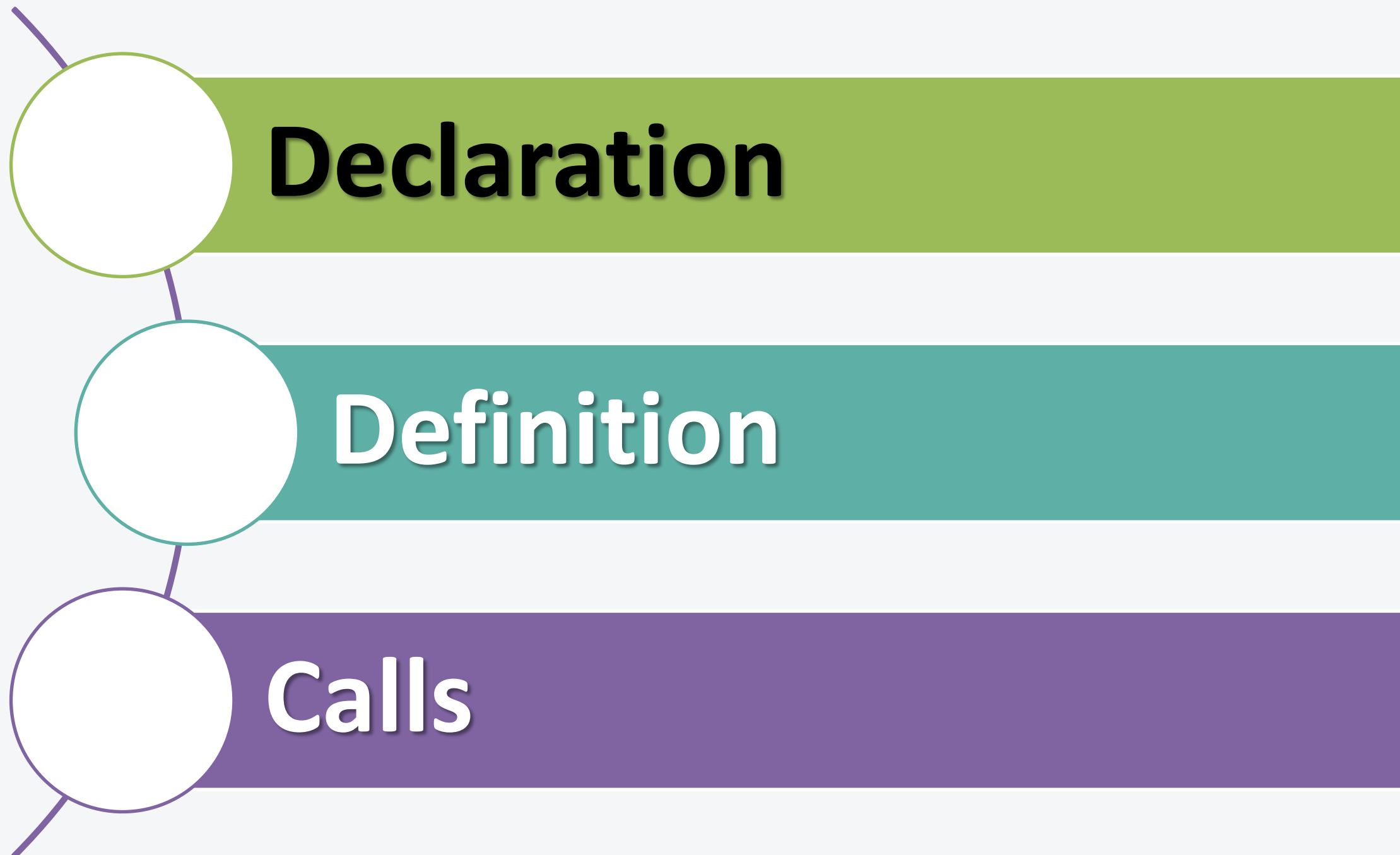
✓ Conceptual Organization of Program

✓ Reduce Program Size

Function

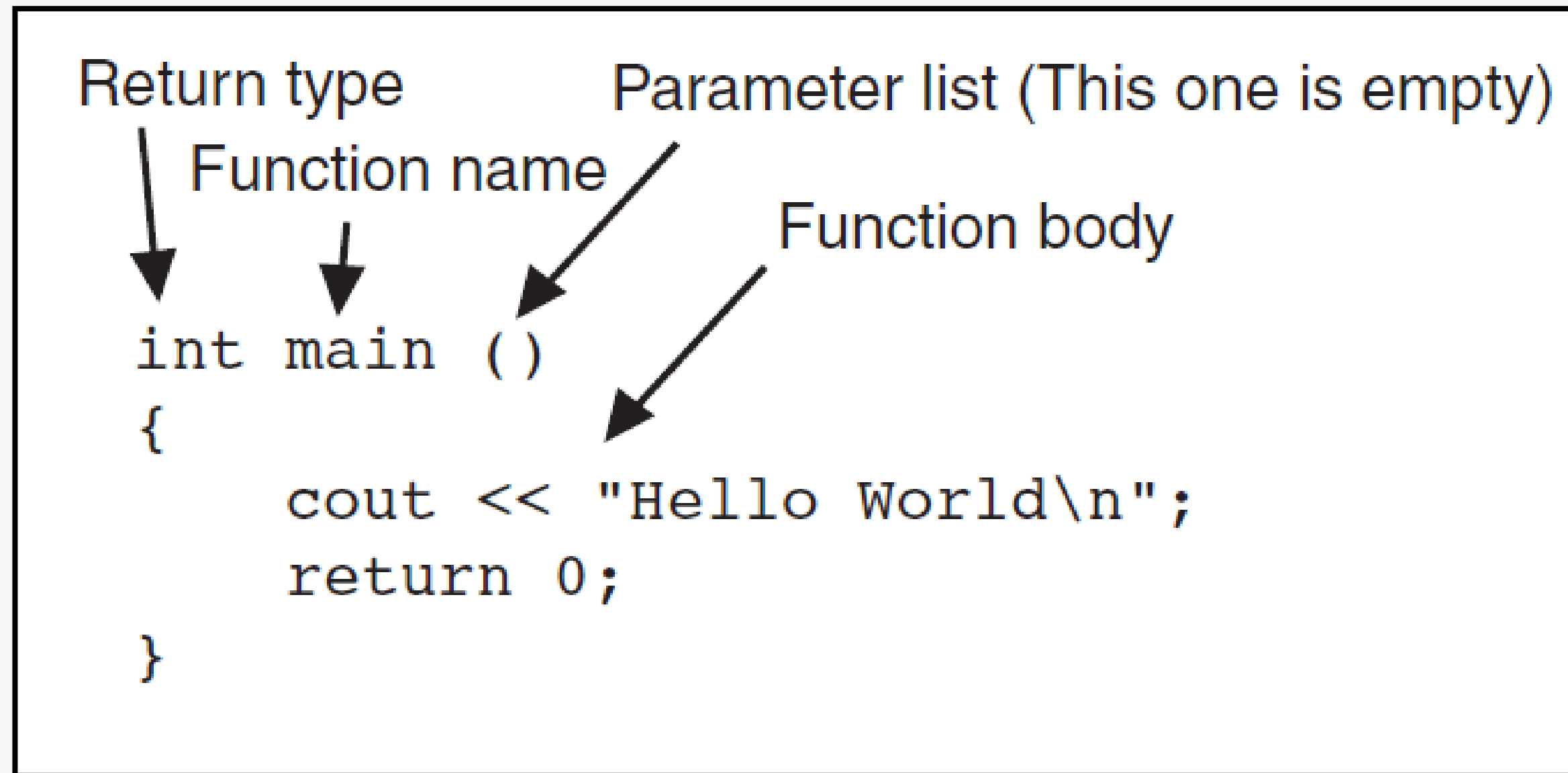
1. A function's single most important role is to
 - a. give a name to a block of code.
 - b. reduce program size.
 - c. accept arguments and provide a return value.
 - d. help organize a program into conceptual units.

Function



Definition

A function definition contains the statements that make up the function.



The line in the definition that reads `int main()` is called the function header .

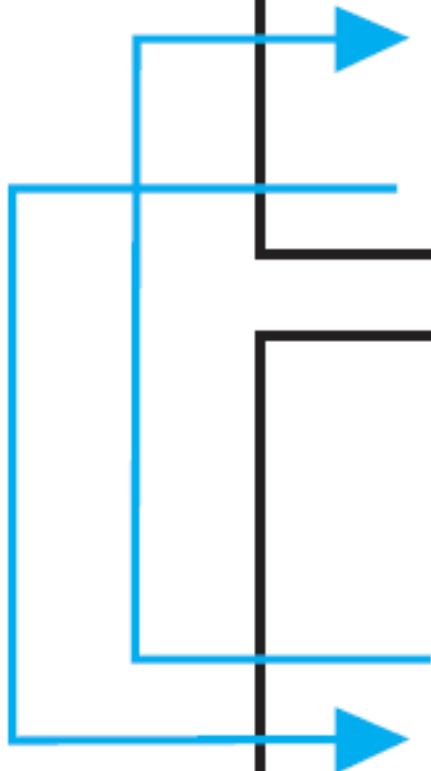
Function Call

- A function call is a statement that causes a function to execute.
- Function main is called automatically when a program starts, but all other functions must be executed by function call statements.
- When a function is called, the program branches to that function and executes the statements in its body.

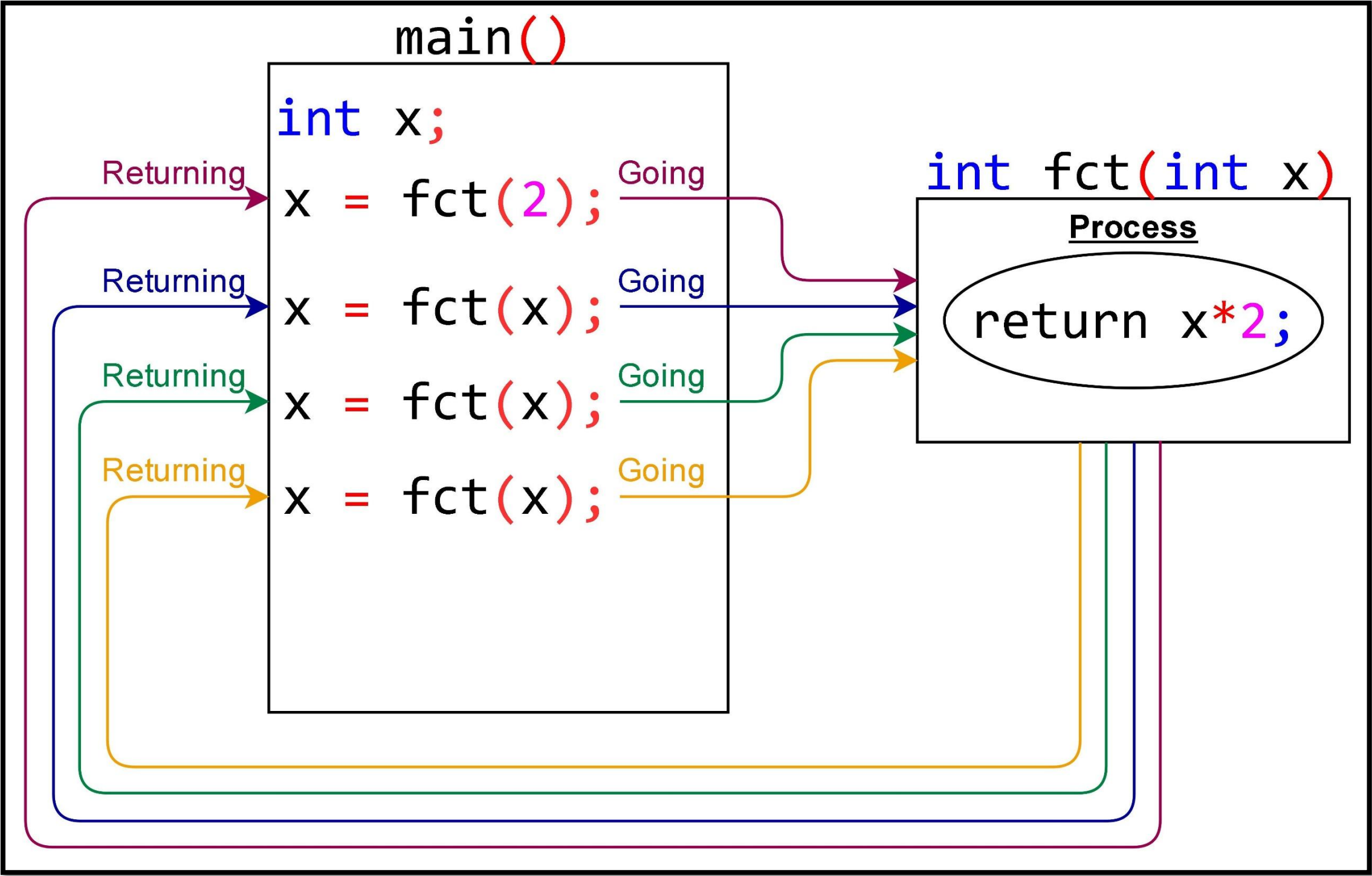
Function Call

```
void displayMessage()  
{  
    cout << "Hello from the function displayMessage.\n";  
}
```

```
int main()  
{  
    cout << "Hello from main.\n"  
    displayMessage();  
    cout << "Back in function main again.\n";  
    return 0;  
}
```

A blue line with arrows illustrates the function call. It starts from the left, splits into two horizontal branches. The upper branch has an arrow pointing to the opening curly brace of the displayMessage() function. The lower branch has an arrow pointing to the displayMessage() call inside the main() function.

Function Call



Function Prototype

- Before the compiler encounters a call to a particular function, it must already know **the function's return type, the number of parameters it uses, and the type of each parameter.**
- One way of ensuring that the compiler has this information is to place the function definition before all calls to that function.
- Another method is to declare the function with a function prototype .

Function Prototype

```
#include <iostream>
using namespace std;

// Function Prototype
void displayValue(int);

int main()
{
    cout << "I am passing several values to displayValue.\n";
    displayValue(5);    // Call displayValue with argument 5
    displayValue(10);   // Call displayValue with argument 10
    displayValue(2);    // Call displayValue with argument 2
    displayValue(16);   // Call displayValue with argument 16
    cout << "Now I am back in main.\n";
    return 0;
}

/*****
 * Definition of function displayValue.
 * It uses an integer parameter whose value is displayed.
 *****/

void displayValue(int num)
{
    cout << "The value is " << num << endl;
}
```

Names in the Declaration

```
void display_point(int, int);
```

```
void display_point(int horiz, int vert);
```


Function

Indicate which of the following is the function prototype, the function header, and the function call:

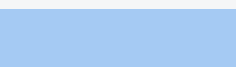
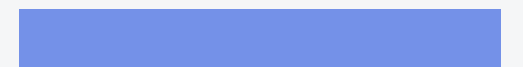
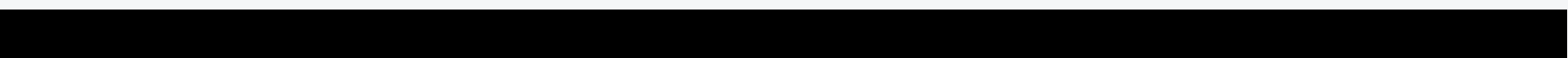
```
void showNum(double num)
```

```
void showNum(double);
```

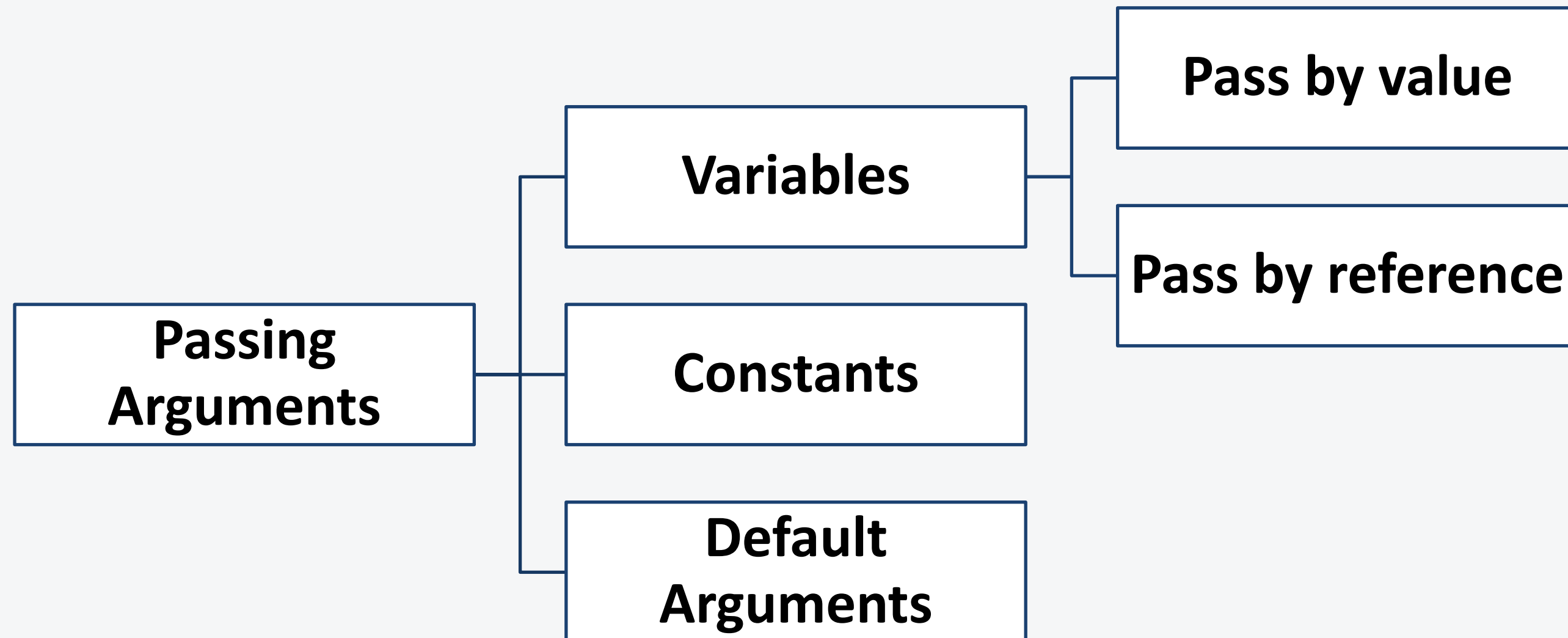
```
showNum( 45.67 );
```

02.

Passing Arguments



Passing Arguments to Function



Passing Constants as Arguments

```
void displayValue(int);

int main()
{
    cout << "I am passing several values to displayValue.\n";
    displayValue(5);    // Call displayValue with argument 5
    displayValue(10);   // Call displayValue with argument 10
    displayValue(2);    // Call displayValue with argument 2
    displayValue(16);   // Call displayValue with argument 16
    cout << "Now I am back in main.\n";
    return 0;
}

/*****
 * Definition of function displayValue.
 * It uses an integer parameter whose value is displayed.
 *****/

void displayValue(int num)
{
    cout << "The value is " << num << endl;
}
```

Passing Variables as Arguments

```
void showSum(int, int, int);

int main()
{
    int value1, value2, value3;

    // Get three integers.
    cout << "Enter three integers and I will display ";
    cout << "their sum: ";
    cin >> value1 >> value2 >> value3;

    // Call showSum passing three arguments.
    showSum(value1, value2, value3);

    return 0;
}

/*****
 * Definition of function showSum.
 * It uses three integer parameters whose sum is displayed.
 *****/

void showSum(int num1, int num2, int num3)
{
    cout << (num1 + num2 + num3) << endl;
}
```

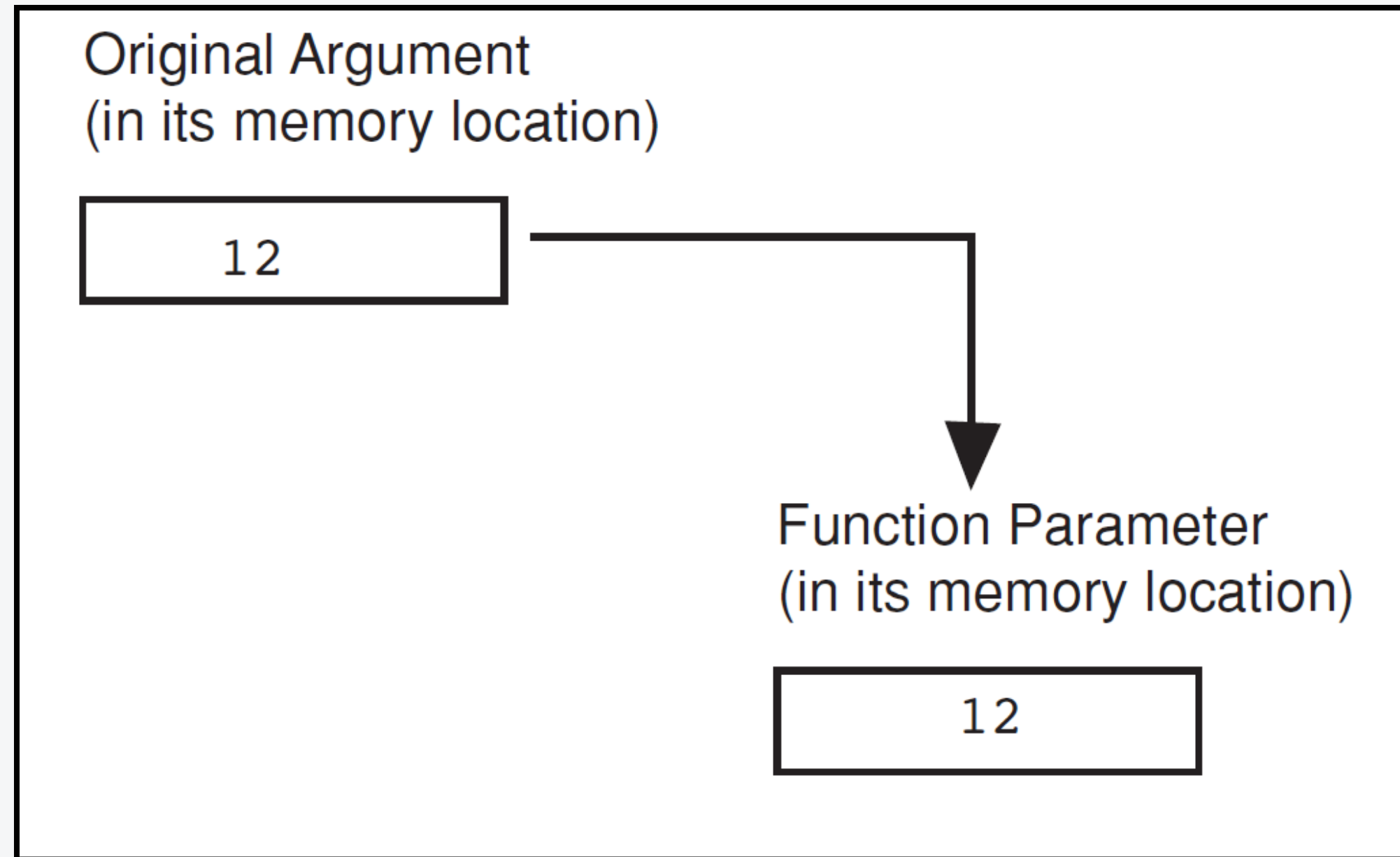
Passing Variables as Arguments

Function Call → `showSum(value1, value2, value3)`

```
graph TD; A["Function Call → showSum(value1, value2, value3)"] --> B["void showSum(int num1, int num2, int num3)"]; A --> C["cout << (num1 + num2 + num3) << endl;"]; A --> D["}"];
```

```
void showSum(int num1, int num2, int num3)
{
    cout << (num1 + num2 + num3) << endl;
}
```

Passing Variables by Value

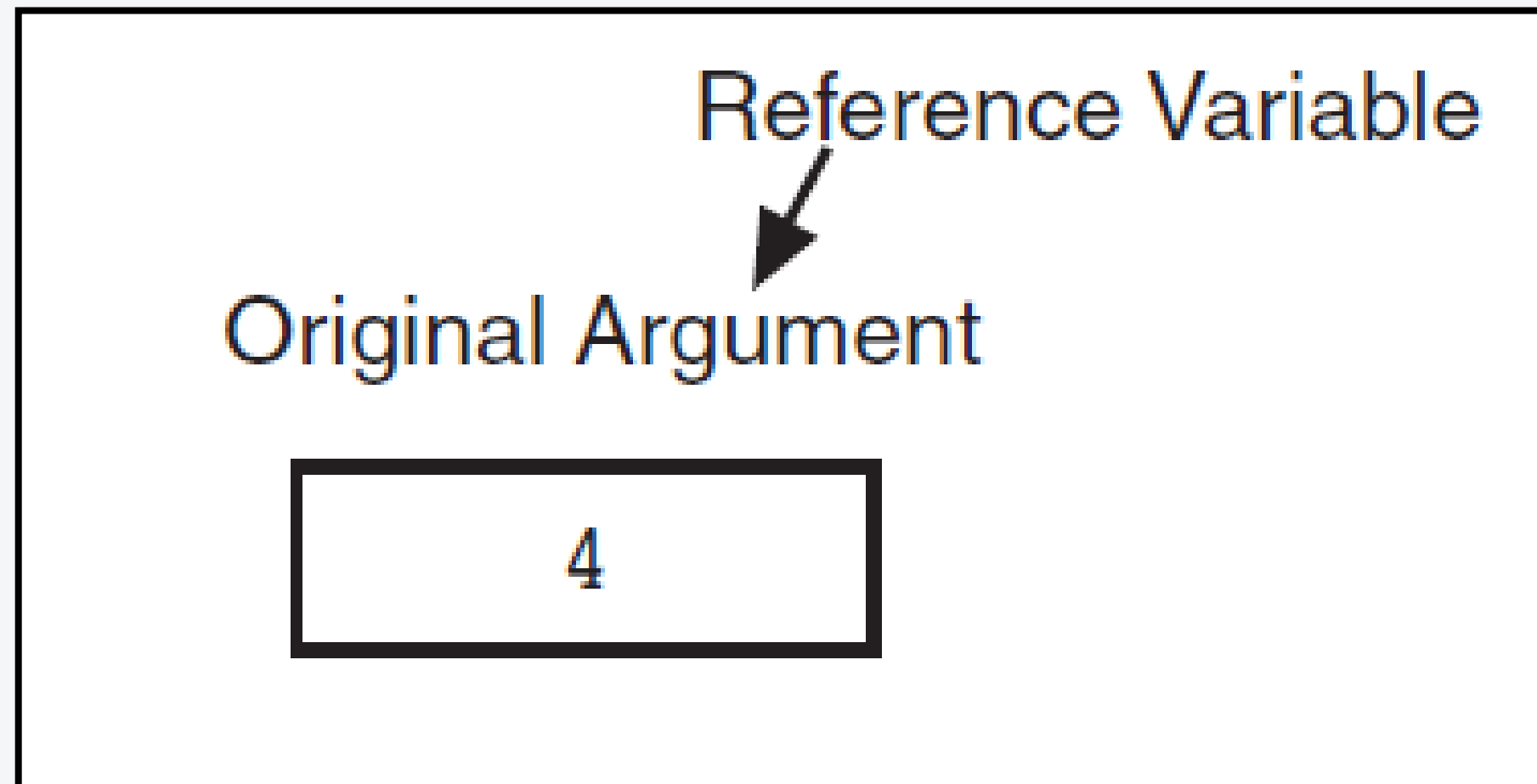


Passing Variables by Value

```
31 void changeMe(int myValue)
32 {
33     // Change the value of myValue to 0.
34     myValue = 0;
35
36     // Display the value in myValue.
37     cout << "Now the value is " << myValue << endl;
38 }
```

```
3  #include <iostream>
4  using namespace std;
5
6  // Function Prototype
7  void changeMe(int);
8
9  int main()
10 {
11     int number = 12;
12
13     // Display the value in number.
14     cout << "number is " << number << endl;
15
16     // Call changeMe, passing the value in number
17     // as an argument.
18     changeMe(number);
19
20     // Display the value in number again.
21     cout << "Now back in main again, the value of ";
22     cout << "number is " << number << endl;
23     return 0;
24 }
```


Passing Variables by Reference



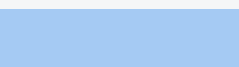
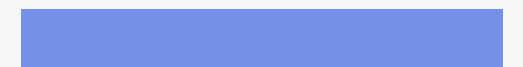
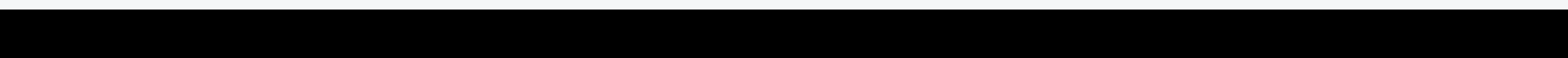
Passing Variables by Reference

```
3  #include <iostream>
4  using namespace std;
5
6  // Function prototype. The parameter is a reference variable.
7  void doubleNum(int &);
8
9  int main()
10 {
11     int value = 4;
12
13     cout << "In main, value is " << value << endl;
14     cout << "Now calling doubleNum..." << endl;
15     doubleNum(value);
16     cout << "Now back in main. value is " << value << endl;
17     return 0;
18 }
```

```
26 void doubleNum (int &refVar)
27 {
28     refVar *= 2;
29 }
```

Task

Write a function called `zeroSmaller()` that is passed two int arguments by reference and then sets the smaller of the two numbers to 0. Write a `main()` program to exercise this function.



Task

- Write a function called swap() that interchanges two time struct passed to it by the calling program.

Read Only Reference Arguments

- Suppose you want to pass an argument by reference for efficiency, but not only do you want the function not to modify it, **you want a guarantee that the function cannot modify it.**
- To obtain such a guarantee, you can apply the const modifier to the variable in the function declaration.

Read Only Reference Arguments

```
void aFunc(int& a, const int& b); //declaration

int main()
{
    int alpha = 7;
    int beta = 11;
    aFunc(alpha, beta);
    return 0;
}

//-----
void aFunc(int& a, const int& b) //definition
{
    a = 107; //OK
    b = 111; //error: can't modify constant argument
}
```

Default Arguments

```
#include <iostream>
using namespace std;

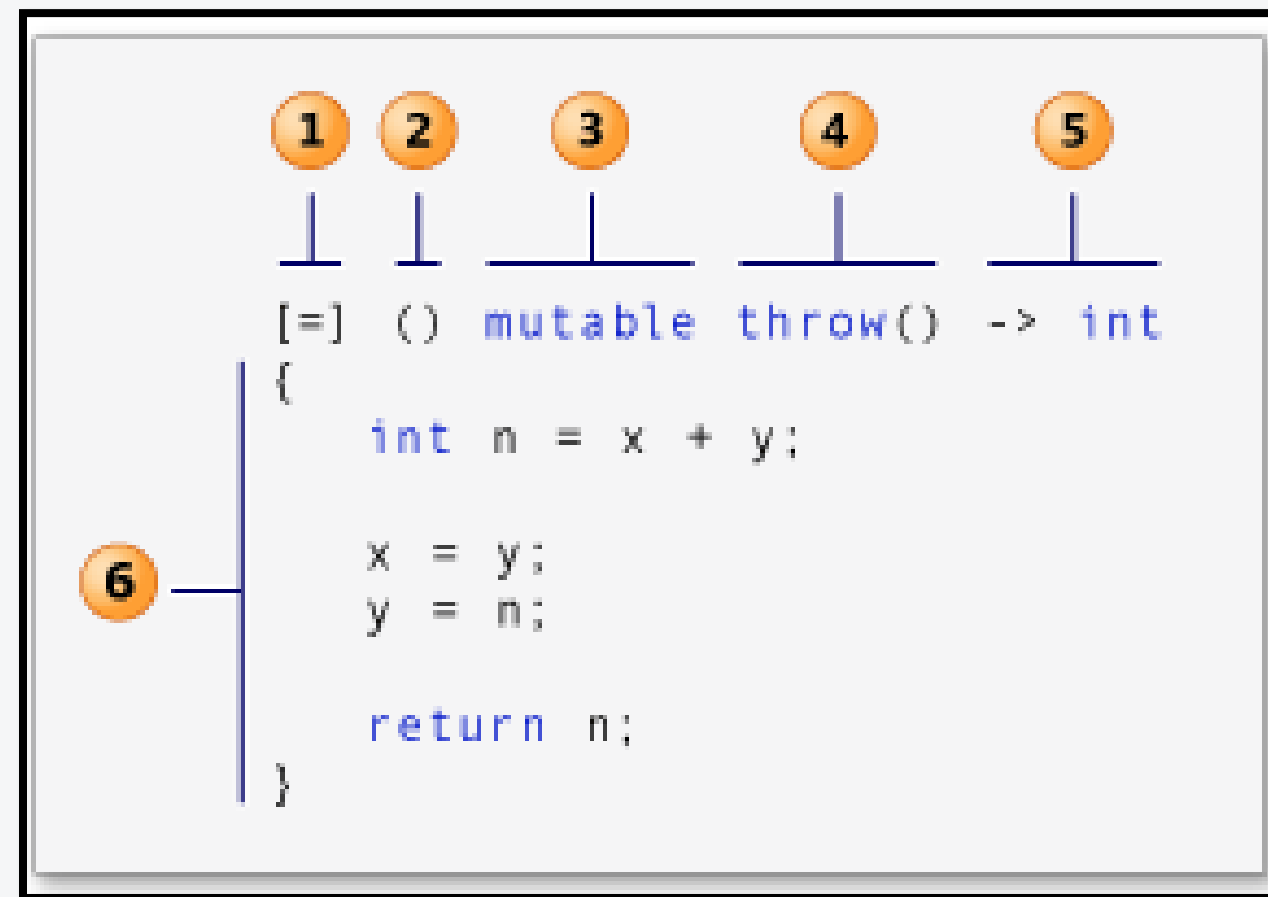
void repchar(char='*', int=45);    //declaration with
                                   //default arguments

int main()
{
    repchar();                    //prints 45 asterisks
    repchar('=');                 //prints 45 equal signs
    repchar('+', 30);             //prints 30 plus signs
    return 0;
}

//-----
// repchar()
// displays line of characters
void repchar(char ch, int n)      //defaults supplied
{                                  //    if necessary
    for(int j=0; j<n; j++)        //loops n times
        cout << ch;              //prints ch
    cout << endl;
}
```

Lambda Expression

- A lambda expression—often called a *lambda*—is a convenient way of defining an anonymous function object (a *closure*) right at the location where it's invoked or passed as an argument to a function.



Lambda Expression

```
int main() {  
    int x = 10;  
    auto show = [x]() {  
        cout << x;  
    };  
  
    show();  
    return 0;  
}
```

```
int main() {  
    int x = 10;  
  
    auto show = [&x]() {  
        cout << x;  
    };  
  
    x = 20; // Change x after the lambda is created  
  
    show();  
    return 0;  
}
```

constexpr

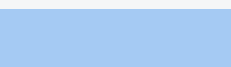
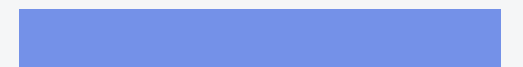
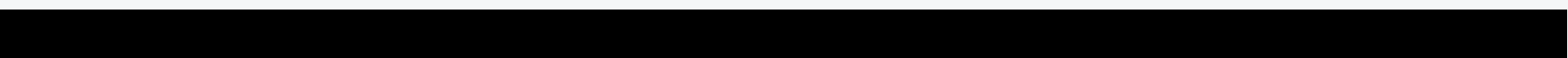
- The constexpr specifier declares that it is possible to evaluate the value of the entities at compile time.
- Such entities can then be used where only compile time constant expressions are allowed

```
constexpr long int fib(int n)
{
    return (n <= 1) ? n : fib(n-1) + fib(n-2);
}

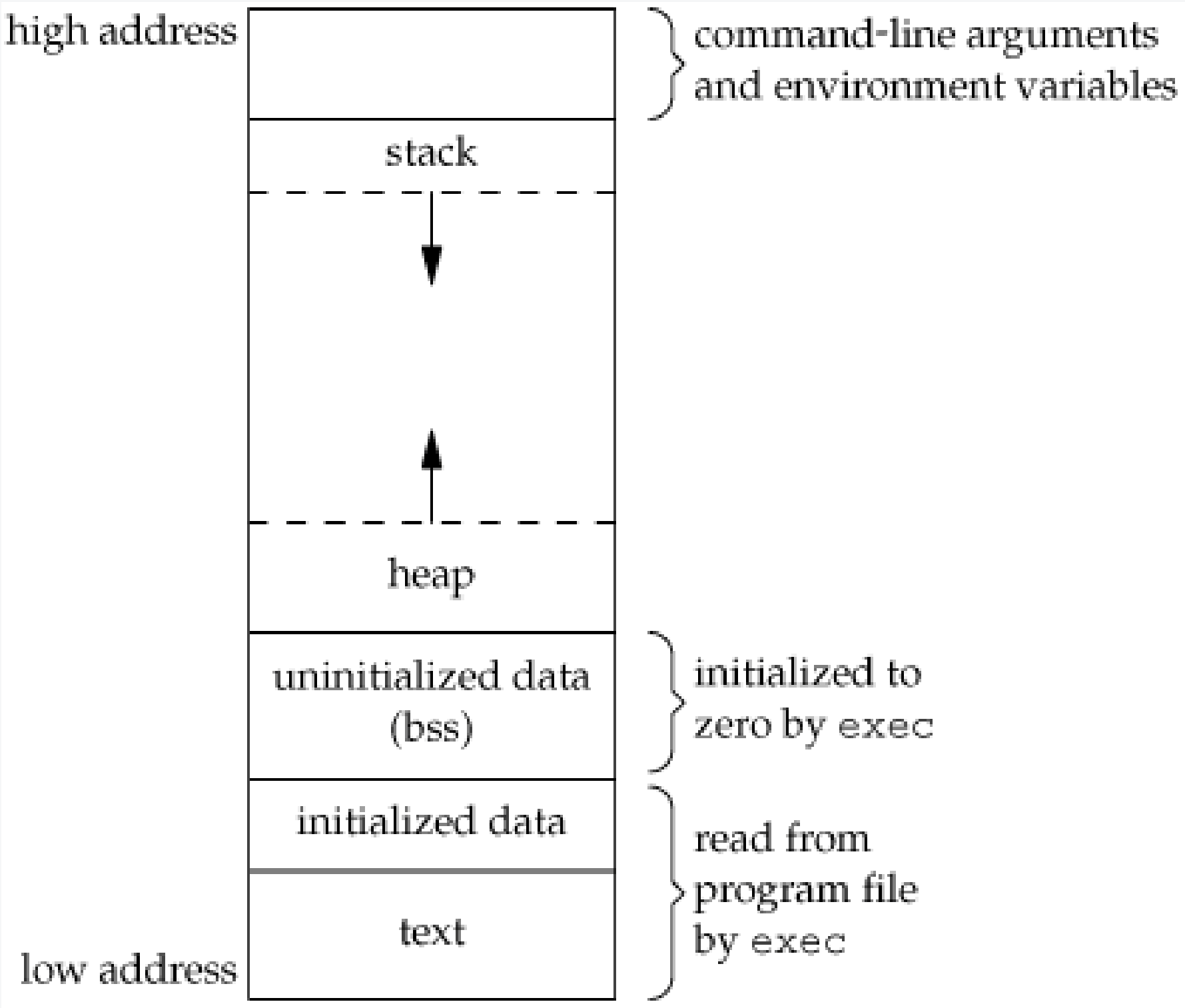
int main ()
{
    // value of res is computed at compile time.
    constexpr long int res = fib(30);
    std::cout << res;
    return 0;
}
```

03.

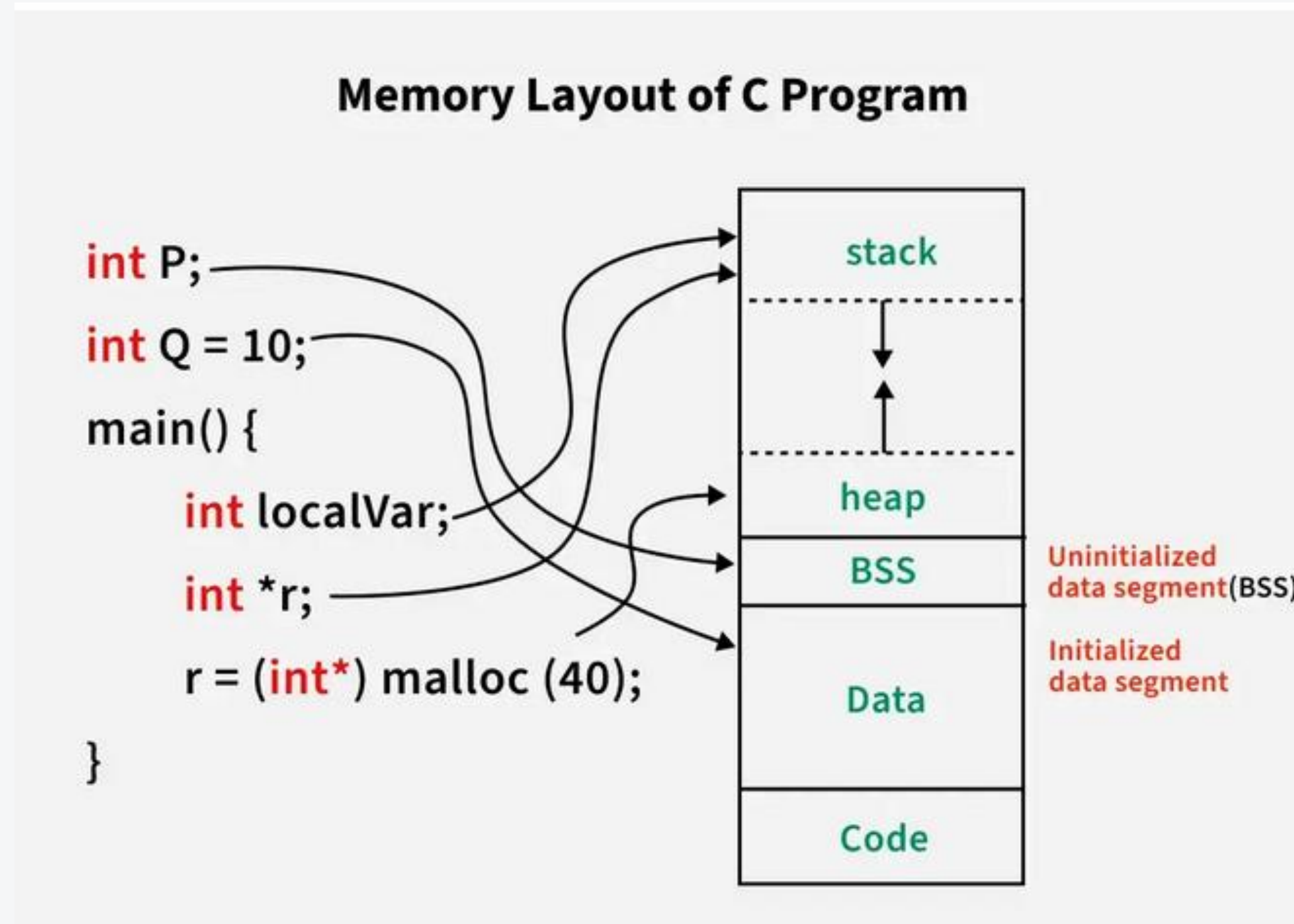
Scope and Storage Class



PROCESS



Memory Layout

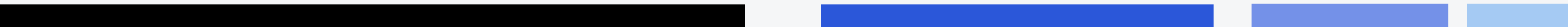


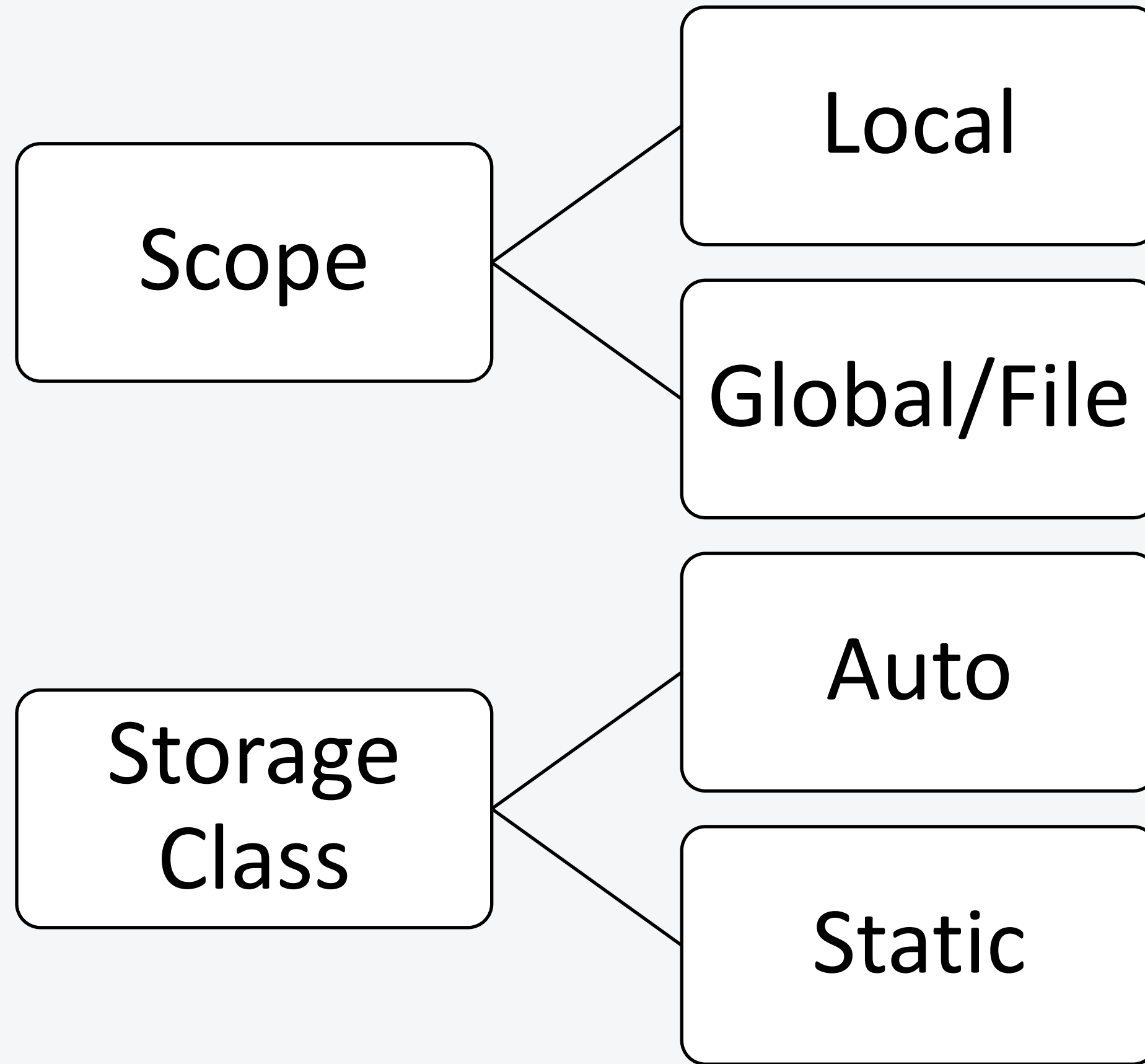
Scope/Visibility

The scope of a variable determines which parts of the program can access it.

Storage Class/Lifetime

The storage class determines how long a variable stays in existence.





Local Variable

- Variables defined within a function body are called local variables.
- Remember main() is also a function.
- Local variables are only visible from within their function.

```
void somefunc()
{
    int somevar;           //variables defined within
    float othervar;        //the function body
    // other statements
}
```


Storage Class of Local Variable

- A local variable is not created until the function in which it is defined is called.
- When control is transferred to the function, variables are created and memory space is set aside for them.
- When the function returns and control is passed back to the calling program, the variables are destroyed and their values are lost.

Local Variable Initialization

- When a local variable is created, the compiler does not try to initialize it.
- It will start off with an arbitrary value, which may be 0 but probably will be something else.

Global/External Variable

- Global variables are defined outside of any function.
- ***A global variable is visible to all the functions in a file.***
- Global variables are also sometimes called external variables, since they are defined external to any function.

Global/External Variable

```
#include <iostream>
using namespace std;
#include <conio.h>           //for getch()

char ch = 'a';              //global variable ch

void getachar();            //function declarations
void putachar();

int main()
{
    while( ch != '\r' )     //main() accesses ch
    {
        getachar();
        putachar();
    }
    cout << endl;
    return 0;
}
```

```
void getachar()              //getachar() accesses ch
{
    ch = getch();
}
//-----
void putachar()              //putachar() accesses ch
{
    cout << ch;
}
```

Global Variable Storage Class

- Global variables have storage class static, which means they exist for the life of the program.
- Memory space is set aside for them when the program begins, and continues to exist until the program ends.
- You don't need to use the keyword static when declaring global variables.

Initialization of Global Variables

- If a global variable is initialized, this initialization takes place when the program is first loaded.
- If a global variable is not initialized explicitly by the program—for example, if it is defined as `int exvar`; then it is initialized automatically to 0 when it is created.

Can we have a variable whose scope is local, but storage class is static?



Static Local Variables

- A static local variable has the visibility of an automatic local variable.
- However, its lifetime is the same as that of a global variable.
- It doesn't come into existence until the first call to the function containing it. Thereafter it remains in existence for the life of the program.
- Static local variables are used when it's necessary for a function to remember a value when it is not being executed; that is, between calls to the function.

Static Local Variables

```
float getavg(float);           //declaration

int main()
{
    float data=1, avg;

    while( data != 0 )
    {
        cout << "Enter a number: ";
        cin >> data;
        avg = getavg(data);
        cout << "New average is " << avg << endl;
    }
    return 0;
}

//-----
// getavg()
// finds average of old plus new data
float getavg(float newdata)
{
    static float total = 0; //static variables are initialized
    static int count = 0;   // only once per program

    count++;                //increment count
    total += newdata;        //add new data to total
    return total / count;    //return the new average
}
```

Here's some sample interaction:

Enter a number: 10	
New average is 10	← total is 10, count is 1
Enter a number: 20	
New average is 15	← total is 30, count is 2
Enter a number: 30	
New average is 20	← total is 60, count is 3

Static Local Variable Initialization

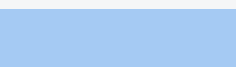
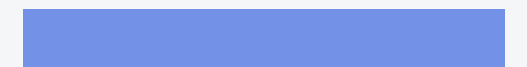
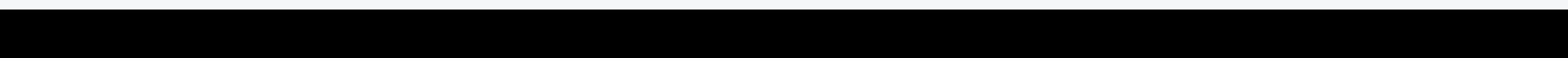
- When static variables are initialized, as total and count are in `getavg()`, the initialization takes place only once—the first time their function is called.
- They are not reinitialized on subsequent calls to the function, as ordinary local variables are.

Task

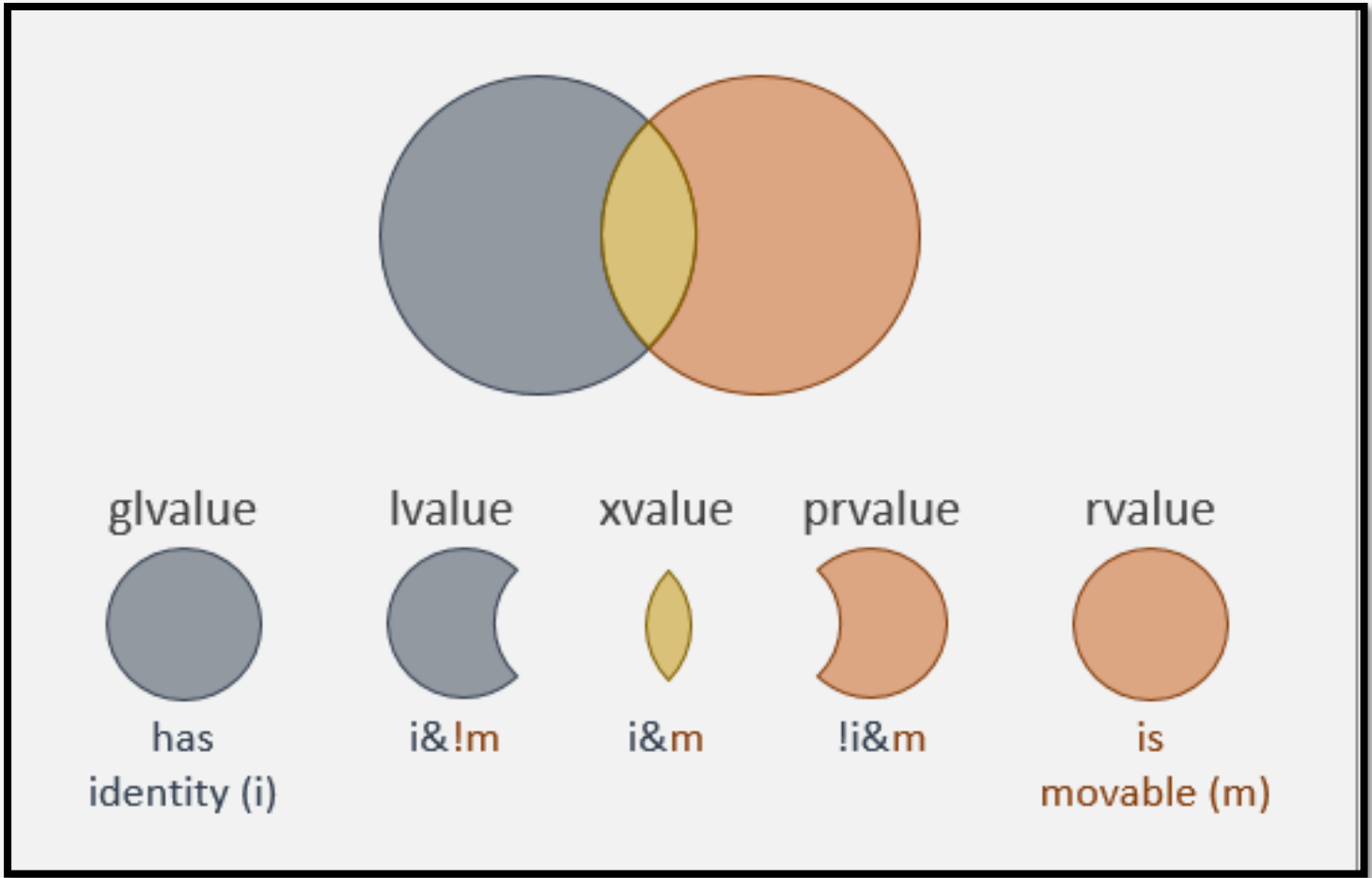
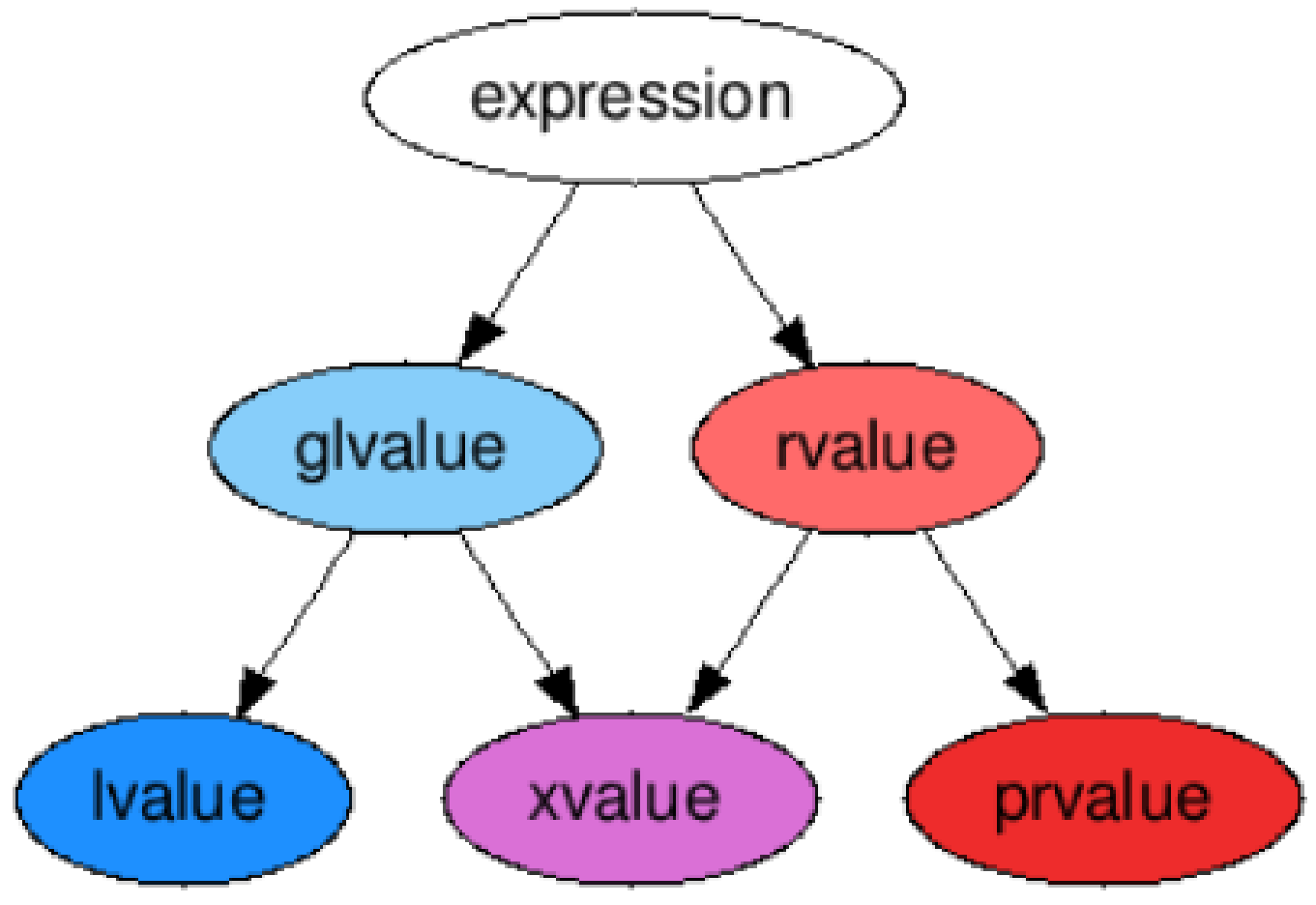
- Write a function that, when you call it, displays a message telling how many times it has been called: “I have been called 3 times”, for instance.
- Write a main() program that calls this function at least 10 times.

04.

Return Statement



Value Categories



void Functions

- It isn't necessary for all functions to return a value.
- A function that does not return anything is called a void function.

```
void displayMessage()  
{  
    cout << "Hello from the function displayMessage.\n";  
}
```

Returning by Value

```
int SUM(int a, int b)
{
    int s1 = a + b;

    // method using the return
    // statement to return a value
    return s1;
}

// Driver method
int main()
{
    int num1 = 10;
    int num2 = 10;
    int sum_of = SUM(num1, num2);
    cout << "The sum is " << sum_of;
    return 0;
}
```

```
#include <iostream>
using namespace std;

// Function to multiply the two
// numbers a and b
auto multiply(int a, int b)
{
    // Return the product
    return a * b;
}

// Driver Code
int main()
{
    int a = 4, b = 5;

    // Function Call
    cout << multiply(a, b);

    return 0;
}
```

Returning by Reference

```
#include <iostream>
using namespace std;
int x;                // global variable
int& setx();          // function declaration

int main()
{
    // set x to a value, using
    setx() = 92;       // function call on left side
    cout << "x=" << x << endl; // display new value in x
    return 0;
}

// -----
int& setx()
{
    return x;          // returns the value to be modified
}
```


Returning by Reference

```
int& setx()  
{  
    int x = 3;  
    return x;  
}
```



Returning by Reference

```
#include <iostream>
using namespace std;

// Function to increment the value
// of a and return the updated value
auto& increase(int& a)
{
    a++;
    return a;
}

// Driver Code
int main()
{
    int b = 10;

    // Function Call
    int& c = increase(b);
    cout << b << '\n'
         << c;

    return 0;
}
```

Returning by Reference

```
#include <iostream>
using namespace std;

// Function to increment the
// value of a
auto increase(int& a)
{
    // Increment a
    a++;

    // Return the updated value
    return a;
}

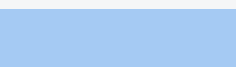
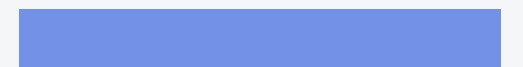
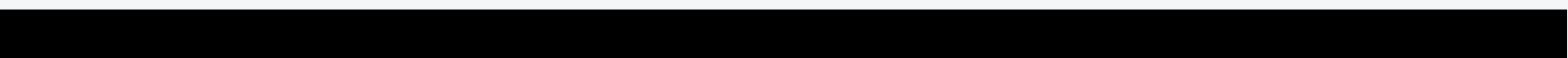
// Driver Code
int main()
{
    int b = 10;

    // Function Call
    int& c = increase(b);

    // Print the value b and c
    cout << b << c;
    return 0;
}
```


05.

Recursion



Recursion

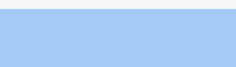
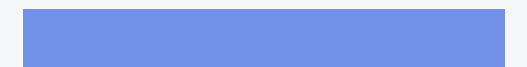
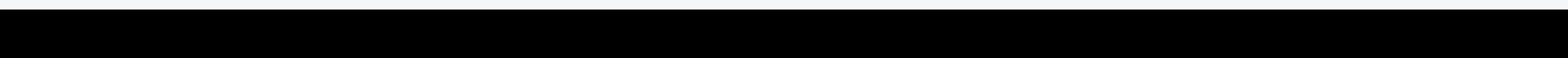
- Recursion involves a function calling itself.
- **Every recursive function must be provided with a way to end the recursion.**
- Each recursion version's variables are stored, but there's only one copy of the function's code.

Recursion

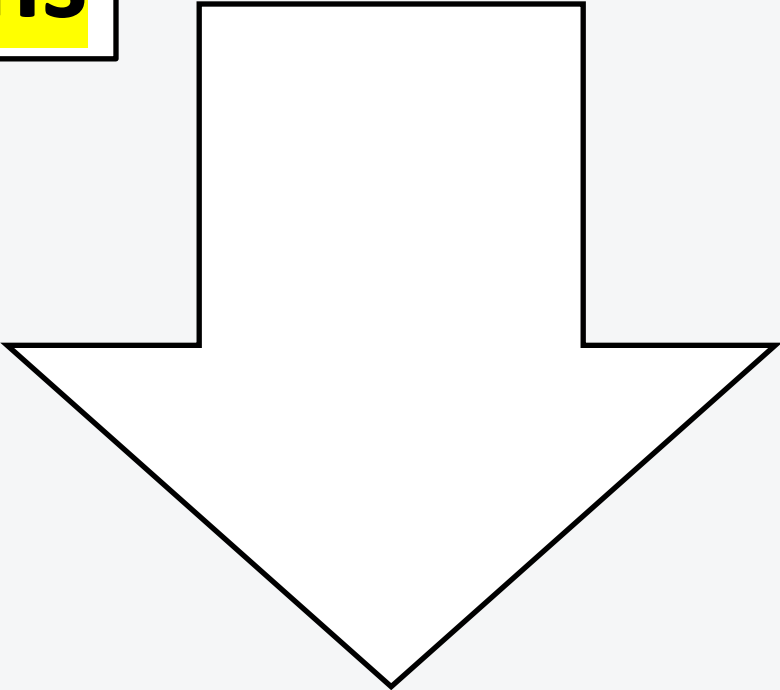
```
unsigned long factfunc(unsigned long n)
{
    if(n > 1)
        return n * factfunc(n-1); //self call
    else
        return 1;
}
```

06.

Inline Functions



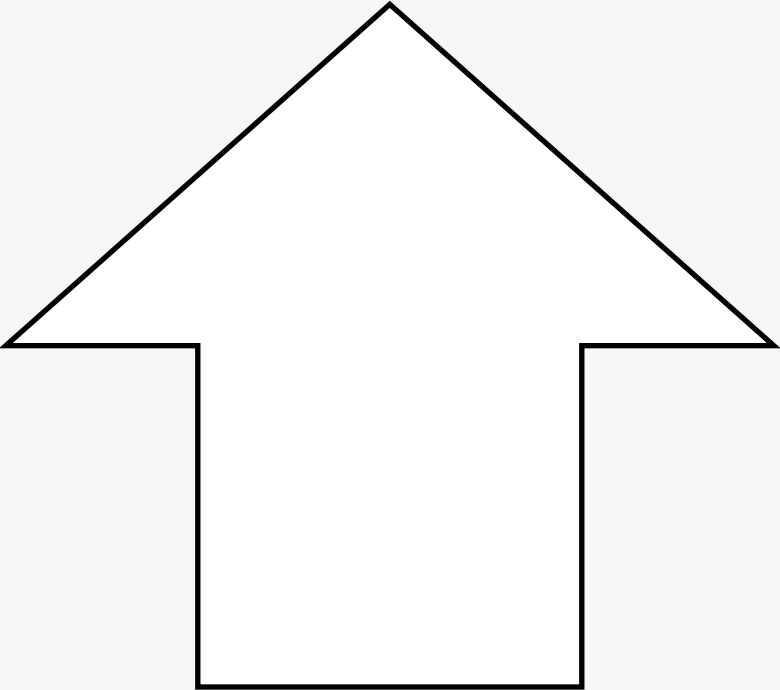
Inline Functions



Memory



Time



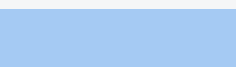
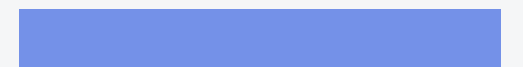
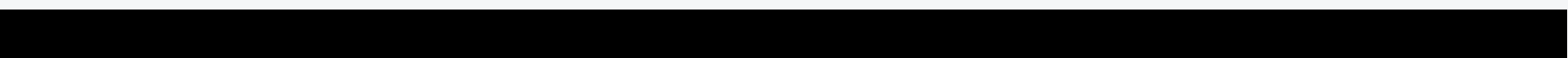
Inline Functions

```
inline float lbstokg(float pounds)
{
    return 0.453592 * pounds;
}
// -----
int main()
{
    float lbs;

    cout << "\nEnter your weight in pounds: ";
    cin >> lbs;
    cout << "Your weight in kilograms is " << lbstokg(lbs)
         << endl;
    return 0;
}
```

07.

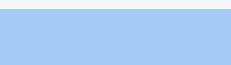
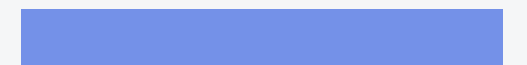
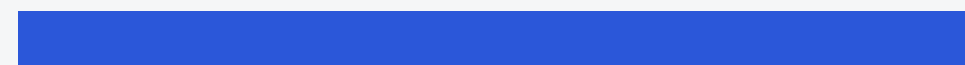
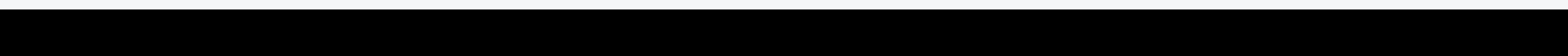
Function Overloading



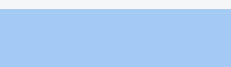
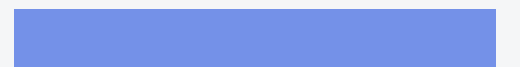
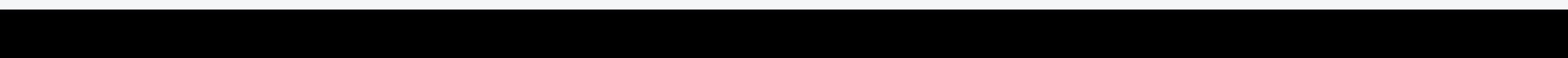
Write a function to add two integers.

Write another function to add two floats.

Write another function to add three floats.



Polymorphism



Different No. of Arguments

```
#include <iostream>
using namespace std;

void repchar();           //declarations
void repchar(char);
void repchar(char, int);

int main()
{
    repchar();
    repchar('=');
    repchar('+', 30);
    return 0;
}

//-----
// repchar()
// displays 45 asterisks
void repchar()
{
    for(int j=0; j<45; j++) // always loops 45 times
        cout << '*';      // always prints asterisk
    cout << endl;
}

//-----
// repchar()
// displays 45 copies of specified character
void repchar(char ch)
{
    for(int j=0; j<45; j++) // always loops 45 times
        cout << ch;       // prints specified character
```

```
        cout << endl;
    }
    //-----
    // repchar()
    // displays specified number of copies of specified character
    void repchar(char ch, int n)
    {
        for(int j=0; j<n; j++) // loops n times
            cout << ch;       // prints specified character
        cout << endl;
    }
```

This program prints out three lines of characters. Here's the output:

```
*****
=====
+++++
```

Different Types of Arguments

```
#include <iostream>
using namespace std;

int add(int a, int b);
double add(double a, double b);

int main() {
    cout << add(10, 2) << endl;
    cout << add(5.3, 6.2);
    return 0;
}

int add(int a, int b) {
    return a + b;
}

double add (double a, double b) {
    return a + b;
}
```

Task

- Raising a number n to a power p is the same as multiplying n by itself p times.
- Write a function `power()` that takes a double value for n and an int value for p , and returns the result as a double. Use a default argument of 2 for p .
- Now, create a series of overloaded functions with the same name that, in addition to double, also work with types `char`, `int`, `long`, and `float`.
- Write a `main()` program that exercises these overloaded functions with all argument types.

**Thank
You!**

